ContractIQ

Phase 1: Motor Claim Document Validator

A practical end-to-end guide: from a customer upload to an auditable validation outcome.

Phase 1 objective	Validate a claim document package before downstream extraction or claim assessment.
Decision boundary	The system never approves or rejects an insurance claim. It gives a reviewable document-validation outcome.
Current implementation	FastAPI service, deterministic content classifier, PDF text extraction, duplicate/missing checks, and structured JSON output.

Audience: business users, claims teams, developers, and reviewers

1. What Phase 1 solves

Motor-claim packages arrive as PDFs, scans, and images. Before an insurer can extract values or assess coverage, it must know whether the right documents were received and whether each upload appears to be the document it claims to be.

Phase 1 is the intake gate. It turns a set of uploaded files into a consistent, machine-readable validation result that a claims officer can understand and act on.

Business questions answered

- Which files were submitted for this claim?

- Are the files supported, non-empty, within the configured size limit, and readable?

- Which document type does the content indicate: RC, policy, licence, claim form, FIR, estimate, invoice, or accident-photo record?

- Does the detected type match the document type expected for that upload?

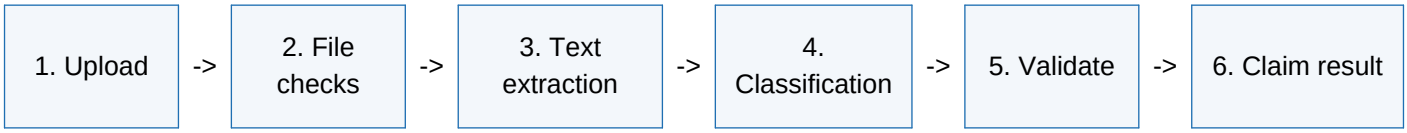
- Are any file contents duplicates? Which required documents are missing?

- Should the package proceed, be corrected, or be sent for human review?

Out of scope in Phase 1

Not performed	Why it is deferred
Claim approval/rejection	A human claim officer remains the decision-maker.
Field extraction	Vehicle number, policy dates, invoice totals, and names are Phase 2 extraction work.
Cross-document comparison	Matching the vehicle number across RC, policy, and invoice is Phase 3.
OCR for scans	Requires an IDP/OCR provider such as Azure Document Intelligence.

2. End-to-end upload flow



Step 1 - Upload

A user opens the local ContractIQ page, enters a claim ID, chooses one or more supported files, and optionally selects the expected document type. The browser sends a multipart request to the versioned API endpoint:

```
POST /api/v1/claims/{claim_id}/validate
```

Supported types are .txt, .pdf, .jpg, .jpeg, and .png. The configured default file-size limit is 10 MB. The file name is displayed in the outcome but is not used to decide the document type.

Step 2 - API intake

The FastAPI route checks that expected-document values, when supplied, line up with the number of files. It reads each upload into an IncomingFile object containing the original name, bytes, and optional expected type. The route delegates to the validation service; it does not embed business rules itself.

Step 3 - Claim-level processing

The validation service processes every file, builds individual file results, checks required-document coverage, and derives a claim-level status. The result is returned as JSON to the browser and is ready for storage or later audit integration.

3. File validation and readability

Check	Current behaviour	Why it matters
Allowed extension	Accepts TXT, PDF, JPG, JPEG, PNG; rejects other extensions.	Prevents unsupported files entering downstream processing.
Empty file	Returns UNREADABLE.	Avoids treating an empty upload as a document.
Size limit	Returns UNREADABLE when over MAX_UPLOAD_SIZE_BYTES.	Controls resource use and reduces abuse risk.
PDF readability	Uses pypdf to open a PDF and extract its embedded text.	Digital PDFs can be classified from their content.
Image / scan	Returns NEEDS_REVIEW with an OCR-required message.	No OCR engine is currently connected.
Exact duplicate	SHA-256 hash identifies identical file bytes.	Avoids double-counting the same upload.

Why an image can be readable to a person but not to Phase 1

A JPG, PNG, or scanned PDF often contains pixels rather than selectable text. The current service deliberately does not guess from the file name. It routes these files for review because genuine content classification needs OCR first. This is expected behaviour, not a claim decision.

Production next step

Connect Azure Document Intelligence or another approved OCR/IDP provider. Its extracted text becomes the input to the same classifier and validation rules. This keeps the business workflow stable while improving coverage for scans and photos.

4. How document classification works

Phase 1 uses a deterministic, explainable content classifier. It lowercases the extracted text, normalizes repeated spaces and line breaks, and searches for document-specific indicators. It never decides solely from a filename such as RC.pdf.

Document type	Examples of content indicators
Registration Certificate (RC)	Certificate of Registration, Registration No, Regn No, Chassis Number, Engine Number, Vehicle Class
Insurance policy	Certificate of Insurance, Policy Schedule, Policy Number, Policyholder, Period of Insurance, Own Damage
Driving licence	Driving Licence / Driving License, Licence Number, DL No, Valid Till, Date of Issue
Claim form	Claim Form, Claimant, Date of Loss, Accident Details, Claim Number
FIR / police report	First Information Report, FIR No, Police Station, Complainant
Garage estimate	Garage Estimate, Repair Estimate, Estimate No, Labour Charges, Estimated Cost
Repair invoice	Tax Invoice, Invoice Number, GSTIN, Bill Amount, Amount Payable
Accident-photo record	Accident Photograph, Damage Photograph, Vehicle Damage Photo

Confidence

Each matched indicator adds to the selected type's score. More evidence raises confidence; close competing types lower it. If confidence is below the configured 0.70 threshold, the service returns NEEDS_REVIEW rather than taking a confident position.

5. Expected-versus-detected validation

Classification identifies what the uploaded content appears to be. Validation compares that result with what the user or workflow expected. This catches a common real-world issue: a customer uploads an RC where the FIR was requested, even if the file name says FIR.pdf.

Expected	Detected from content	Result	Meaning
RC	RC	VALID	The content matches the expected document type.
FIR	RC	WRONG_DOCUMENT	The content is readable but is not the requested type.
None	Driving licence	VALID	Automatic detection succeeded without a pre-selected type.
RC	Unknown / low confidence	NEEDS_REVIEW	Do not reject; send for a human check.
Any	Unreadable / invalid PDF	UNREADABLE	The service cannot safely inspect the content.
Any	Exact same file as earlier	DUPLICATE	The file content was already uploaded.

Important distinction

WRONG_DOCUMENT is not the same as UNREADABLE. Wrong means the content was understood and did not match the requested type. Unreadable means the content could not safely be inspected. NEEDS_REVIEW means the system deliberately asks for human help instead of pretending certainty.

6. Claim-level completeness

A standard motor claim package in the current configuration requires five documents: claim form, RC, policy, driving licence, and garage estimate. FIR, repair invoice, and accident-photo records can still be classified when present but are not mandatory in the default rule set. Actual requirements should later be driven by claim type, policy rules, and incident facts.

Claim result field	Explanation
required_documents	The configured list of documents required for the claim.
missing_documents	Required types that did not receive a VALID document.
documents_received	Number of uploaded files, including duplicates and unreadable files.
valid_documents	Number of file results with status VALID.
invalid_documents	All non-VALID file results in the current Phase 1 response.
overall_status	COMPLETE only when there are no missing documents and no non-VALID uploads; otherwise NEEDS_CORRECTION.
files	An audit-friendly item for every submitted file, including detected type, confidence, message, evidence, and duplicate reference.

Example response


```
{"file_name":"document_04.pdf", "expected_document":"rc", "detected_document":"fir",
"classification_confidence":0.96, "status":"WRONG_DOCUMENT", "message":"Expected rc, but
content indicates fir"}
```

7. Code structure and responsibilities

Location	Responsibility
app/main.py	Creates the FastAPI application, test page, health endpoint, and API router.
app/api/v1/routes/claims.py	Accepts HTTP uploads, validates request shape, and creates IncomingFile objects.
app/services/validator.py	Runs file checks, text extraction, duplicate detection, expected-vs-detected validation, and claim aggregation.
app/services/classifier.py	Defines document indicators and calculates deterministic classification confidence.
app/schemas/documents.py	Defines Pydantic document types, statuses, and response contracts.
app/core/config.py	Reads environment-based settings such as size and confidence thresholds.
tests/test_validator.py	Verifies complete claims, wrong documents, duplicates, ambiguity, RC wording, and scanned-PDF review handling.
sample_data/	Contains clearly labelled synthetic PDFs for local demo and regression testing.

Why this separation matters

The API layer can change without changing validation rules. The classifier can be improved or replaced by a model without changing the response contract. OCR can be introduced as a text-extraction provider without rewriting claim-completeness logic. This is the intended production evolution path.

8. Local testing guide

Start the service

```
$py = 'C:\Users\mital\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe' & $py -m pip install -r requirements.txt & $py -m uvicorn app.main:app --host 127.0.0.1 --port 8001 --reload
```

Use the upload page

- Open http://127.0.0.1:8001/.

- Enter a claim ID, such as CLM-DEMO-001.

- Select one or more synthetic PDFs under sample_data/motor_claim_clm_demo_001/.

- For a package demonstration, leave Expected document type as Detect automatically.

- Click Validate documents and inspect the returned per-file and claim-level JSON.

Run automated tests

```
& $py -m pytest -q
```

Expected result: six tests pass in the committed Phase 1 baseline. The tests are a regression safety net; they should be expanded when new document types, OCR, or policy-specific requirements are introduced.

9. LLMs, OCR, and the roadmap

Capability	Needed now?	Reason
Deterministic validation rules	Yes	Best for required documents, duplicates, file checks, auditability, and predictable expected-vs-detected comparisons.
LLM	No	Not required for the current classification and validation workflow. It can add cost and non-determinism if used where rules are enough.
OCR / IDP	Required for scans	Needed to convert image-only PDFs, JPGs, and PNGs into text before they can be classified.
Azure Document Intelligence	Phase 1 production enhancement	A suitable managed OCR/IDP option for document text, layout, and later structured fields.
LLM-assisted extraction	Later, selectively	Useful for varied layouts, nuanced explanations, and complex unstructured evidence after deterministic controls are in place.

Recommended next phases

- Phase 1.1: OCR integration plus encrypted/malformed-PDF handling and an operator review queue.

- Phase 2: Extract structured fields from classified documents, with field confidence and source evidence.

- Phase 3: Cross-document checks: registration number, owner/insured name, policy period, and invoice consistency.

- Phase 4: Policy coverage rules and human decision support. Do not autonomously settle claims.

- Production hardening: identity and access control, malware scanning, object storage, database audit trail, secret management, monitoring, rate limits, CI/CD, and data-retention controls.

Key takeaway

Phase 1 is a reliable document-intake gate: it accepts files, inspects their readable content, classifies them with transparent rules, compares them with what was expected, identifies duplicates and gaps, and returns a human-reviewable outcome. OCR extends its reach to scans; an LLM is optional and belongs later, where unstructured understanding genuinely adds value.